

Computer Security

Course Code: CSE-429

**Term Paper on Network Intrusion Detection
Models: from classical to advanced techniques**

Name: Faiyaz Kamal
ID: 202014075
Section: CSE-22(A)

Contents

Abstract	2
1 Introduction	3
2 Historical Context: Why Intrusion Detection Emerged	4
3 Classical Models of Intrusion Detection	5
3.1 Signature-Based Detection	5
3.2 Statistical Anomaly Detection	6
3.3 Specification-Based Detection	6
4 Hybrid and Ensemble Approaches	8
5 Machine Learning–Based Models	9
5.1 Supervised Learning	9
5.2 Unsupervised and Semi-Supervised Learning	9
5.3 Feature Engineering and Pipeline Concerns	9
6 Deep Learning for Intrusion Detection	11
6.1 Autoencoders	11
6.2 Sequence Models	11
6.3 Convolutional Approaches	11
7 Case Studies and Tools	13
8 Current Challenges	14
8.1 Encrypted Traffic	14
8.2 Adversarial Evasion	14
8.3 Dataset Limitations	14
9 Future Directions	15
9.1 Federated and Continual Learning	15
9.2 Self-Healing Systems	15
9.3 Lightweight Models at the Edge	16
10 Conclusion	17
References	18

Abstract

Intrusion Detection Systems (IDS) are the listening posts of a network. While firewalls control doors, IDS listens to conversations, patterns, and timing, then warns when something feels wrong. This paper traces the evolution of network intrusion detection from early rule lists and basic statistics to modern machine learning and deep neural architectures. We explain how each family of models works, where it shines, and where it struggles. We also discuss encrypted traffic, adversarial evasion, dataset pitfalls, and the practical side of running IDS in real environments. The goal is readable clarity with technical depth: short, plain sentences when possible; longer, threaded paragraphs when needed. Figures summarize core pipelines, and a comparison table highlights trade-offs. The paper closes with near-term directions such as federated training and self-healing response loops that tie detection to action without losing human oversight.

Chapter 1

Introduction

A network is a busy city. Messages cross paths, queues form, shortcuts appear, and every so often someone tries to slip past the guards. Classic perimeter tools stop obvious threats, but they cannot watch everything at once. This gap is where IDS lives.

We focus on **Network-based IDS (NIDS)**, which analyzes packets or flows in transit, and we contrast it—when useful—with **Host-based IDS (HIDS)**, which inspects logs, system calls, or process behavior on endpoints. The scope remains network-centric because traffic features, sequence patterns, and protocol context are the raw ingredients most NIDS models digest.

Three guiding questions shape the discussion:

1. *What is the model looking for?* Known patterns, deviations from a baseline, or violations of a specification.
2. *How does it learn?* By rules, by thresholds, or by data-driven training (supervised, unsupervised, semi-supervised).
3. *How does it hold up in practice?* Throughput, latency, false alerts, maintenance effort, and how it copes with change.

We will return to these questions at the end of each major section, so the thread stays visible as the methods get more advanced.

Chapter 2

Historical Context: Why Intrusion Detection Emerged

The idea that we can watch “normal” behavior and flag oddities is older than the modern internet. Anderson’s 1980 report on audit trails pointed to patterns in access logs as warning signs. In the late 1990s, DARPA’s offline evaluations seeded a common reference point for IDS researchers. The KDD’99 dataset soon followed. It offered scale and labels, both useful; yet it also froze the threat mix in time and encouraged models that memorized artifacts rather than learned resilient signals.

Meanwhile, the real network changed—more users, more protocols, and more encryption. Rule lists caught many attacks with crisp precision. But new attacks kept showing up. That tension—precision on the known versus coverage on the unknown—anchors the evolution from signature rules to anomaly models and, later, to learning systems that generalize.

Chapter 3

Classical Models of Intrusion Detection

Classical models do not try to be clever. They try to be clear. Their value is easy triage, stable performance, and efficient computation on high-speed links.

3.1 Signature-Based Detection

A signature is a compact pattern that points to a known threat: a byte sequence in a payload, a suspicious header field, a specific handshake sequence, or a logic rule combining these. Snort popularized the approach through readable rules and a large community of contributors. When a packet or a flow matches a rule, the engine raises an alert with a reason that operators can understand.

Strengths. Accuracy on known threats is high, and the false positive rate is usually low. Rule updates can be pushed quickly, and explainability is strong: the system tells you which signature matched and why.

Limits. Anything novel can pass unseen. Rule sets grow quickly and must be curated. Attackers also mutate payloads, re-order steps, or pad bytes, so brittle signatures need frequent refresh.

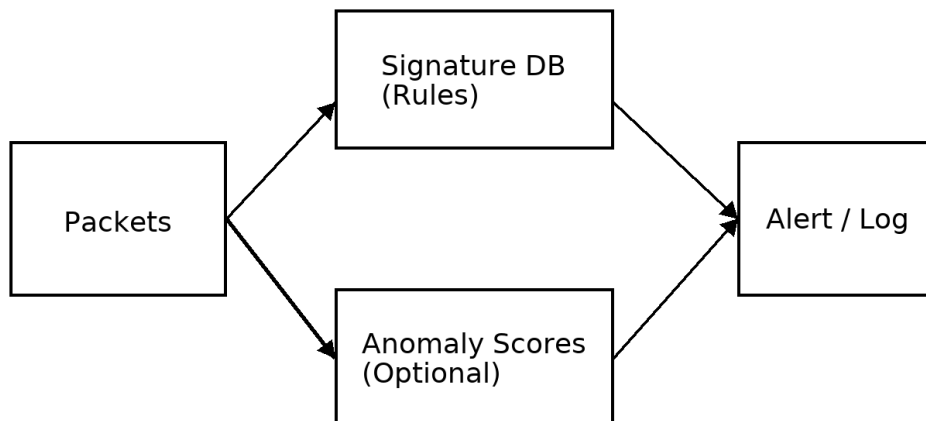


Figure 3.1: Signature and anomaly paths feeding a common alert sink. Left: packets; middle: rule checks and optional anomaly scores; right: alert/log output for SOC pipelines.

How to read Figure 1. Packets enter on the left. One path checks rules; another computes scores from baselines. Either path can trigger an alert that lands in the same place: logs, a SIEM index, or a message bus for incident tooling. Many production deployments use the rule path on high priority and the anomaly path as a second opinion to reduce blind spots.

3.2 Statistical Anomaly Detection

Instead of matching a fingerprint, anomaly detection builds a picture of “normal” and focuses on deviations. Basic versions keep rolling averages of bytes per second, connection counts, or flow durations. More careful versions estimate distributions, tail behavior, or correlations between fields. If normal traffic is a melody, anomalies are off-key notes that pop out when you listen long enough.

Strengths. It can catch surprises, including zero-days and slow scans. It adapts over time and does not require labels.

Limits. Normal is not fixed. Morning traffic differs from midnight traffic. Product launches, outages, and seasonality all sway the baseline. Without context or dampening, alerts can spike. Tuning matters.

3.3 Specification-Based Detection

Here we do not guess what “normal” looks like; we *write it down*. Protocol fields must follow a spec; a service must stay within allowed states and transitions; an IoT device may only talk to a short list of peers. Anything that strays is suspect.

Strengths. Clear rules help in regulated settings and embedded systems. Alerts are specific: which field, which rule, what value.

Limits. Modern microservices and rapid releases shift behavior often. Specifications lag and exceptions pile up. Writing and maintaining crisp specifications across a moving system is hard work.

A Quick Comparison

Method	What it looks for	Where it shines	Main risk
Signature-based	Known byte/logic patterns	Low false positives; clear reasons	Blind to new variants
Statistical anomaly	Deviations from baseline	Finds surprises; no labels needed	Alert noise if drifting
Specification-based	Violations of allowed behavior	Strong in constrained systems	Maintenance burden

Table 3.1: Classical IDS families at a glance.

Chapter 4

Hybrid and Ensemble Approaches

No single model covers everything. A practical answer is layering. One common pattern runs signatures in front, anomaly scores in parallel, and a correlation step that requires either high rule confidence or repeated anomalies before paging anyone. Another pattern uses an ensemble of light classifiers—trees, linear models, one-class detectors—and takes a vote. Ensembles smooth out quirks of any one model and reduce sensitivity to small shifts in traffic.

The operational theme is simple: push cheap, understandable checks early; reserve heavier models for traffic that is suspicious or important; record enough context so that analysts can replay events without guessing.

Chapter 5

Machine Learning–Based Models

Machine learning added two promises: learn richer patterns from data and reduce manual rule churn. Real networks delivered two reminders: labels are expensive, and distribution shift is constant.

5.1 Supervised Learning

Supervised models train on labeled flows or sessions. Common choices include Decision Trees, Random Forests, Gradient Boosting, and Support Vector Machines. With curated datasets like UNSW-NB15 and CIC-IDS2017, offline scores can look excellent. In production, mislabels, stale features, and unseen traffic produce a different story. Good pipelines handle class imbalance, provide calibration for scores, and expose top features that drove a decision.

Feature examples. Flow duration; packet counts; mean and variance of inter-arrival time; TCP flag patterns; byte entropy; unusual ports; ratios such as inbound/outbound bytes.

Training loop in short. Split by time (not random) so evaluation mirrors the future. Track precision, recall, F1, and alert volume per hour. Keep a rolling holdout window to catch drift. Store the model, the code that made it, and the exact feature schema.

5.2 Unsupervised and Semi-Supervised Learning

When labels are scarce, clustering and outlier detectors help. k -means clusters similar flows; DBSCAN marks dense clusters and isolates stray points; Isolation Forests score how quickly a flow becomes separable. One-class SVM learns the contour of “normal” and flags points outside it. Semi-supervised methods train on normal-only data, which is more available, and look for reconstructions or distances that exceed a threshold.

Where it works. Branch offices, IoT segments, or specialized subnets where traffic is stable. The model learns the rhythm and pings when a new beat arrives.

Where it struggles. Busy cores with wide, fast-changing mixes. Cluster boundaries blur; thresholds wander.

5.3 Feature Engineering and Pipeline Concerns

A good IDS is as much plumbing as math. Features must be computed fast, consistently, and with grace under packet loss. Encoders should be deterministic; scaling factors need a clear

source; missing values cannot crash the stream. Think of the pipeline as a factory line: raw packets on one side, usable features on the other, all logged just enough that an investigation can retrace steps.

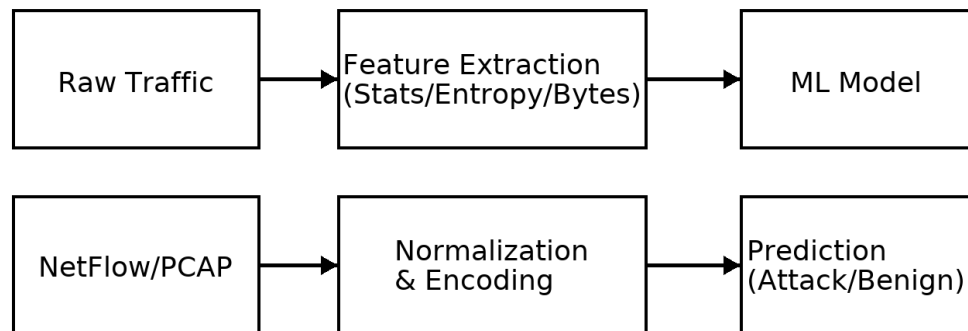


Figure 5.1: Typical ML workflow: raw traffic to features to a trained model and final predictions. Each box implies code and tests, not just an idea.

Reading Figure 2. The left column reminds us that inputs differ: raw PCAP, NetFlow, or summarized records. The middle emphasizes feature creation and normalization. The right ties a model to a prediction service. Between the boxes you should imagine contracts: schemas, versioning, and basic sanity checks.

Chapter 6

Deep Learning for Intrusion Detection

Deep models replace many hand-crafted features with learned representations. They also demand careful handling: stable training, sensible input shapes, and honest tests against drift and noise.

6.1 Autoencoders

Autoencoders learn to compress and reconstruct inputs. Train them on benign traffic; they reconstruct benign flows well. Malicious flows reconstruct poorly and produce higher errors. The threshold sets sensitivity. Add sparsity in the hidden layer to force compact codes. For streaming, mini-batch training and periodic refresh help maintain shape as traffic evolves.

6.2 Sequence Models

Attacks often unfold as sequences: scan, probe, exploit, beacon, exfiltrate. RNNs, LSTMs, and GRUs read ordered events and build memory of context. They work well on NetFlow sequences per source, per destination, or per session window. Padding and masking are required for variable-length sequences. Attention helps the model focus on relevant steps without drowning in long histories.

6.3 Convolutional Approaches

Treat payload bytes or flow matrices as images. Convolutions pick up local motifs such as unusual n-grams or rare header combinations. With 1D CNNs on byte streams, the model stays fast and hardware-friendly. With 2D CNNs on flow-feature grids, it learns spatial relations between features without manual pairing.

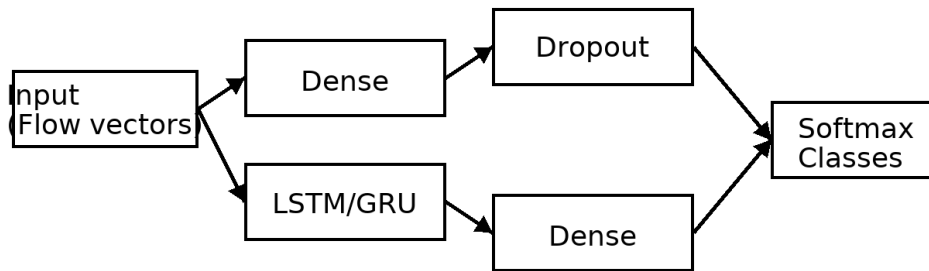


Figure 6.1: Layers in modern deep IDS: dense blocks, dropout for regularization, recurrent units for temporal context, and a softmax head for class scores.

Training pragmatics. Balance classes; mix easy and hard negatives; monitor validation loss and real alert volume; keep a tiny rule set nearby to catch blatant threats while the model warms up.

Chapter 7

Case Studies and Tools

Snort. A rule engine with human-readable signatures and a rich ecosystem. Many organizations start here because it is transparent, lightweight, and well documented. Rule authorship teaches precise thinking about threats.

Suricata. Multithreaded from the ground up, with powerful packet decoding and extensibility. It fits high-throughput links and plays well with Lua scripting and external classification hooks.

Zeek (Bro). Not a signature engine but a network analysis framework. It parses protocols deeply and emits structured logs (HTTP, DNS, TLS, SSH). These logs become the backbone for detection, hunting, and model features. Zeek is a good partner for ML: it distills traffic into semantically rich events.

A practical deployment often uses all three: Zeek for context, a rule engine for sharp detections, and an ML service for the gray areas. Glue them with a message bus (Kafka), store results in a search index (Elasticsearch), and let a SIEM or a homegrown dashboard tie alerts to tickets.

Chapter 8

Current Challenges

8.1 Encrypted Traffic

Payload visibility shrinks as TLS becomes the default. You still have metadata: SNI, JA3/JA4 fingerprints, certificate chains, timing, sizes, and directionality. These carry signal, especially for command-and-control patterns and odd client fingerprints. But privacy rules constrain collection. The design task is to do the most with the least: fewer fields, stronger models, tighter retention.

8.2 Adversarial Evasion

Once models decide access, attackers start shaping inputs. Padding, reordering, and tiny perturbations can shift a score across a threshold. Defenses include adversarial training (mix perturbed samples into training), randomized smoothing (add noise so small changes matter less), and ensemble diversity (so a trick for one model fails on another). It is not a single fix; it is layered friction.

8.3 Dataset Limitations

Public corpora are valuable, but they age. Labels can be noisy, and synthetic traffic lacks the lived-in variability of production. A healthier pattern is continuous, internal curation: capture small, rotating slices with privacy filters; annotate critical incidents; and replay them through the pipeline. Treat datasets as living assets, not one-time downloads.

Chapter 9

Future Directions

9.1 Federated and Continual Learning

Many organizations see the same threats but cannot share raw data. Federated learning trains a shared model by sending updates instead of packets. Continual learning refreshes the model over time with small, careful steps. Together they promise broader coverage without breaking trust. Success hinges on stable update schedules, drift monitors, and rollback paths.

9.2 Self-Healing Systems

Detection alone is not the finish line. Pair it with orchestration: block a source, rate-limit a path, force re-authentication, or quarantine a segment. Keep a human in the loop for big moves, but automate the small ones to cut response time. The art is choosing actions that help more than they hurt.

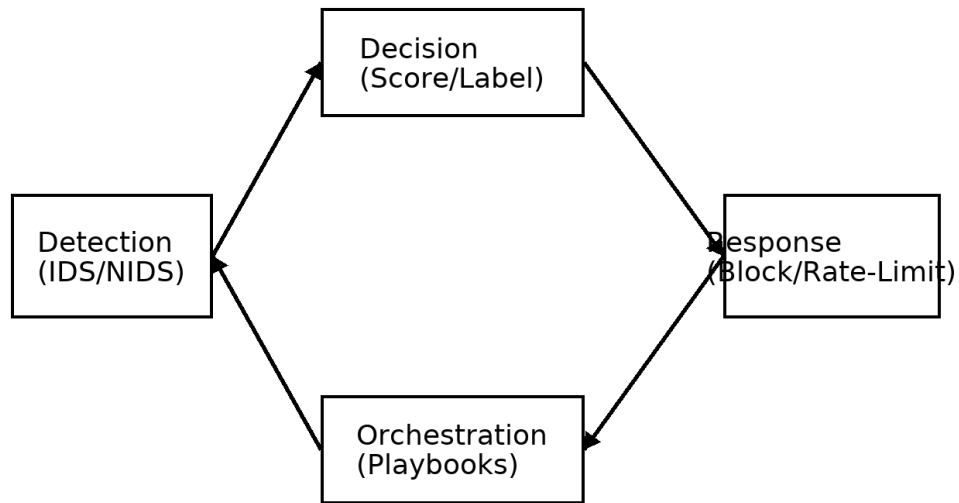


Figure 9.1: Closed loop: detection produces a decision; orchestration applies a response; the system observes results and adjusts policies.

9.3 Lightweight Models at the Edge

Branch routers and IoT gateways cannot host heavy stacks. Compact models—tiny CNNs, distilled trees, or rules enhanced with learned thresholds—push detection closer to the source. Edge hints, sent upstream, improve central models without flooding links.

Chapter 10

Conclusion

The story repeats: clarity, then coverage, then scale. Rules offer clarity. Anomalies offer coverage. Learning offers scale—if we accept maintenance: data pipelines, retraining, drift checks, and cooperation between tools and people. A dependable NIDS is not a single algorithm; it is a well-tended garden of methods that keep each other honest.

References

- Anderson, J. P., *Computer Security Threat Monitoring and Surveillance*, 1980.
- Lippmann, R. P., et al., “The 1999 DARPA Off-Line Intrusion Detection Evaluation,” *Computer Networks*, 2000.
- Tavallaei, M., et al., “A Detailed Analysis of the KDD Cup 99 Dataset,” IEEE CISSDA, 2009.
- Ring, M., et al., “A Survey of Network-Based Intrusion Detection Data Sets,” *Computers & Security*, 2019.
- Sommer, R. and Paxson, V., “Outside the Closed World: On Using Machine Learning for Network Intrusion Detection,” *IEEE S&P*, 2010.
- Kim, J., et al., “Long Short-Term Memory Recurrent Neural Network Classifier for Intrusion Detection,” ICPTS, 2016.
- Cisco Talos, *Snort User Manual*, Cisco Systems, 2023.
- Moustafa, N. and Slay, J., “UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems,” *MILCOM*, 2015.
- Sharafaldin, I., Lashkari, A. H., and Ghorbani, A. A., “Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization,” *CIC-IDS2017*, 2018.